

sAIfer Lab

Joint lab on Safety and Security of AI

Py-01 · Introduzione a Python

Basi di informatica, gestione dati e statistica



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAIfer Lab

Joint lab on Safety and Security of AI

Cos'è Python

Concetti di base, caratteristiche, installazione, codice sorgente

Concetti di base

- Linguaggio di programmazione
- Codice sorgente
- Programma
- Interprete
- Compilatore
- Python

Linguaggio di programmazione

- **Linguaggio formale** usato per scrivere istruzioni che un computer può eseguire
- Rappresenta in forma strutturata i processi computazionali
- Permette di tradurre il codice in **operazioni comprensibili ed eseguibili da una macchina**

Codice sorgente

- Testo di un programma scritto in un linguaggio di programmazione **leggibile dagli esseri umani** (Python, C, Java)
- Contiene le istruzioni che descrivono il comportamento del programma

Programma

- Un **programma** (o applicazione, spesso abbreviato in **app**) è un insieme organizzato di istruzioni scritte in un linguaggio di programmazione
- Una volta eseguito da un computer, svolge uno **specifico compito** o risolve un determinato problema

Interprete e compilatore

Interprete

Traduce ed esegue il codice sorgente **istruzione per istruzione**, durante l'esecuzione

Esempio: Python

Compilatore

Traduce **l'intero codice sorgente in anticipo** in linguaggio macchina o in un codice intermedio

Genera un file eseguibile o oggetto

L'esecuzione avviene dopo, senza bisogno del codice sorgente originale

Esempio: C

Python: a cosa serve

Linguaggio di **alto livello**, interpretato, interattivo e orientato agli oggetti: dai piccoli script ai programmi molto complessi

- Applicazioni desktop: interfacce grafiche (GUI) e strumenti da riga di comando (CLI)
- Analisi matematiche e scientifiche dei dati, con grafici dei risultati
- Applicazioni web
- Amministrazione di sistemi e automazione di attività
- Attività DevOps

Perché Python (1/2)

- **Sintassi leggibile** e adatta ai principianti
- **“Batteries included”**: la libreria standard include moduli per file, networking, matematica, data e ora, test e molto altro
- **Grande comunità e molte risorse**: tutorial, video, forum e meetup locali per ogni argomento e livello

Perché Python (2/2)

- **Ampia diffusione** nel mondo reale
- **Versatile e scalabile**
- **Gratuito e multipiattaforma**
- **Open source**: Python Software Foundation License Version 2
  - Ampi diritti di utilizzo, modifica e distribuzione, anche in software proprietari
  - Chiunque può contribuire allo sviluppo

Caratteristiche chiave

Interpretato

Il codice viene tradotto ed eseguito istruzione per istruzione

Multiparadigma

Permette diversi stili: orientato agli oggetti, imperativo, funzionale

Tipizzazione dinamica

Il tipo è associato ai valori, non alle variabili: si controlla in esecuzione, non si dichiara

Fortemente tipizzato

Non consente operazioni non sicure tra tipi incompatibili senza segnalarle

Installare Python

- Esistono diversi modi per installare Python e vari strumenti per scrivere il codice
- La scelta dipende da preferenze, sistema operativo e obiettivi di sviluppo

Thonny

Gratuito, multipiattaforma e semplicissimo da installare: https://thonny.org/

Non adatto a progetti complessi, ma **ottimo per un corso introduttivo**

Ambienti di sviluppo (IDE) e distribuzioni

- PyCharm (www.jetbrains.com/pycharm): completo e professionale, per principianti e progetti complessi
- Anaconda (www.anaconda.com): diffuso per l'analisi dei dati, include Spyder e JupyterLab
- Wing IDE (www.wingware.com): leggero e professionale, dedicato a Python

Per installazione e configurazione: documentazione ufficiale per il proprio sistema operativo

Strumenti online

Utilizzabili subito, **senza installazione**

- **Google Colab** (colab.research.google.com): immediatamente pronto all'uso, basta un account Google
- Altri servizi online (cercate "python online")

Codice sorgente in Python

- Un semplice **file di testo** con estensione `.py`
- Un programma può essere costituito da **uno o più file** di codice sorgente
- Possono esserci altri file con risorse utili al programma: immagini, icone e altro

Commenti

- Porzioni di testo nel codice **ignorate dall'interprete** durante l'esecuzione
- Descrivono il funzionamento del codice e ne facilitano la comprensione
- Chiariscono perché il codice è stato scritto in un certo modo

```
# Questo è un commento!
```



sAIfer Lab

Joint lab on Safety and Security of AI

Variabili

Assegnazione, nomi, tipi numerici, booleani, stringhe

Variabili

- In Python le variabili sono **nomi associati a un oggetto**
- Contengono un **riferimento** (puntatore) all'indirizzo di memoria in cui l'oggetto è memorizzato
- Dopo l'assegnazione, si accede all'oggetto tramite il nome della variabile

Assegnazione

- L'**assegnazione** associa un valore a una variabile
- Si usa l'operatore `=`: **non indica uguaglianza matematica**, ma associa un nome a un oggetto in memoria

```
x = 10
nome = "Alice"
pi_greco = 3.14
```

`x` ⟶ intero `10`

`nome` ⟶ stringa "Alice"

`pi_greco` ⟶ numero `3.14`

Cosa significa `x = x + 1`?

```
x = x + 1
```

- Si calcola prima il **lato destro**: il valore attuale di `x` più 1
- Poi si associa il risultato al nome `x`
- Non è un'equazione: è un **aggiornamento**

Assegnazione composta

Aggiornare una variabile partendo dal suo valore attuale è molto frequente: Python offre una forma abbreviata

```
x += 1      # equivale a x = x + 1
x -= 2      # equivale a x = x - 2
x *= 3      # equivale a x = x * 3
x /= 4      # equivale a x = x / 4
```

- Si scrivono **senza spazi** tra i due simboli: `+=`, non `+ =`

La variabile contiene un riferimento

- In Python la variabile **non contiene direttamente il dato**: memorizza un riferimento alla posizione in memoria del dato
- Distinzione sottile ma rilevante: la approfondiremo più avanti
- Per semplicità si dice "**la variabile contiene il valore x**", anche se tecnicamente non è corretto

Nomi delle variabili (1/2)

- Il nome dovrebbe indicare il **significato** del valore: `lista_utenti`, `numero_giorni`, `importo_totale`
- Lunghezza qualsiasi; caratteri ammessi: lettere `A-Z a-z`, cifre `0-9`, underscore `_`
- Lettere accentate (es. `età`) accettate, ma è preferibile evitarle

Nomi delle variabili (2/2)

- Il primo carattere **non può essere una cifra**: deve essere una lettera o un underscore
- Per ora non useremo nomi che iniziano con underscore
- Niente spazi o caratteri speciali come **!£$#%**

Maiuscole e minuscole contano

- Python è **case sensitive**: `nome` e `Nome` sono due variabili diverse
- `true` non è il valore booleano `True`

```
nome = 'Ada'
print(Nome)      # NameError: name 'Nome' is not defined
x = true         # NameError: name 'true' is not defined
```

Nomi leciti e non leciti

Nomi leciti

```
a
a122
lista_utenti
un_elemento
_a1          # da evitare per ora
__a2         # da evitare per ora
unElemento   # lecito, non PEP 8
ABC          # PEP 8: COSTANTI
```

Nomi non leciti

```
3aa
valore!
somma totale
```

Variabili: convenzioni

- Oltre alle regole obbligatorie esistono **convenzioni** che è bene adottare (PEP 8)
- Nomi in minuscolo, parole separate da underscore (`snake_case`): `importo_totale`
- Costanti tutte maiuscole: `MAX_VALORE`

peps.python.org/pep-0008/#function-and-variable-names
peps.python.org/pep-0008

Tipi numerici

- Python supporta numeri interi (`int`), in virgola mobile (`float`) e complessi (`complex`)
- Gli **operatori** rappresentano operazioni: addizione, sottrazione, moltiplicazione, divisione…

docs.python.org/it/3/library/stdtypes.html#numeric-types

Operatori aritmetici

```
5 + 3      # addizione
5 - 3      # sottrazione
5 * 3      # moltiplicazione
5 / 3      # divisione
5 // 3     # divisione intera (floor division):
           # arrotonda verso il basso
5 % 3      # modulo: resto della divisione
5 ** 3     # elevamento a potenza
```

La divisione / restituisce sempre un float

```
6 / 2       # 3.0 (non 3)
7 / 2       # 3.5
6 // 2      # 3: floor division, intero
            # se gli operandi sono interi
```

- Con un `int` e un `float`, il risultato è un `float`:

```
2 * 1.5     # 3.0
3 + 2.0     # 5.0
```

Precedenza degli operatori

Come in matematica, le operazioni non vengono eseguite da sinistra a destra, ma secondo una **precedenza**:

1. `**` (potenza)
2. `*`, `/`, `//`, `%`
3. `+`, `-`

- A parità di precedenza si procede **da sinistra a destra**
- Le **parentesi** permettono di cambiare l'ordine

Parentesi

```
2 + 3 * 4              # 14
(2 + 3) * 4            # 20

(-b + d) / (2*a)       # corretto
-b + d / 2*a           # ERRATO: vale -b + (d / 2) * a
```

- Attenzione: `-3**2` vale `-9`, perché la potenza viene calcolata prima del segno meno
- Scrivere `(-3)**2` per ottenere `9`

Booleani

- Due soli valori: `True` e `False`; il loro tipo, `bool`, è una sottoclasse di `int`
- Per questo `True` si comporta come 1 e `False` come 0

```
print(True + True)    # 2
print(True == 1)      # True
```

```
2
True
```

Valore di verità

- I booleani esprimono il **valore di verità** di un'espressione

```
a = True
print(a)
a = False
print(a)
```

```
True
False
```

Assegnazione e confronto

`=` assegnazione

Associa un valore a una variabile

`==` confronto

Il risultato è `True` o `False`

```
x = 10
y = 20
answer = x == y
```

- `x == y` produce `False`, che viene assegnato alla variabile `answer`

Confronti: esempi (1/2)

```
a = 2 < 5
print(a)
a = 4 > 10
print(a)
a = 4 <= 3
print(a)
a = 3 >= 3
print(a)
```

```
True

False

False

True
```

Confronti: esempi (2/2)

```
# attenzione: confronto vs assegnazione
a = 5 == 4+2
print(a)
a = 5 == 4+1
print(a)
a = 6 != 9
print(a)
# or: vedi più avanti
a = (5 == 4) or (5 == 4+1)
print(a)
```

```
False

True

True


True
```

docs.python.org/it/3/library/stdtypes.html#boolean-type-bool

Operatori di confronto

Operatore	Significato	Esempio	Risultato
`<`	minore	`2 < 5`	`True`
`>`	maggiore	`4 > 10`	`False`
`<=`	minore o uguale	`4 <= 3`	`False`
`>=`	maggiore o uguale	`3 >= 3`	`True`
`==`	uguale	`5 == 4`	`False`
`!=`	diverso	`6 != 9`	`True`

- `=` assegna, `==` confronta: non confonderli
- Si scrivono `<=` e `>=`, non `=<` o `=>`

Operatori logici

Combinano valori booleani (ad esempio i risultati di più confronti) e producono `True` o `False`

- `a and b` è `True` solo se **entrambi** sono `True`
- `a or b` è `True` se **almeno uno** dei due è `True`
- `not a` **inverte** il valore

Esempio: `eta >= 18 and eta < 65` è `True` se l'età è compresa tra 18 e 64

Tabella di verità

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

- Precedenza: prima `not`, poi `and`, infine `or`; nel dubbio, parentesi: `a or (b and c)`
- Short-circuit: se il primo operando determina già il risultato, il secondo non viene valutato

Stringhe

- Sequenze di caratteri definite usando virgolette **singole**, **doppie** o **triple**
- L'operatore **+** **concatena** le stringhe

```
a = 'hello'
b = "world!"

c = a+b
print(c)
```

```
helloworld!
```

docs.python.org/it/3/library/stdtypes.html#text-sequence-type-str

Apostrofi e virgolette nelle stringhe

- Se la stringa contiene un apostrofo, va racchiusa tra virgolette **doppie**
- Se contiene virgolette doppie, tra virgolette **singole**

```
s = 'l'albero'          # ERRORE: la stringa finisce dopo 'l'
# SyntaxError: unterminated string literal

s = "l'albero"          # corretto
t = 'disse "ciao"'      # corretto
```

La funzione `len()`

- Restituisce la lunghezza della stringa: il **numero di caratteri**
- Diversamente dal C, non esiste un carattere nascosto di fine stringa
- Gli spazi contano!

```
s = 'hello world'
print(len(s))
```

```
11
```

Per ora pensiamo a `len()` come a un comando; più avanti vedremo cosa significa "funzione"

Il metodo `upper()`

- `upper()` restituisce una **nuova stringa** in maiuscolo

```
s1 = 'Hello World!'
s2 = s1.upper()

print(s1)
print(s2)
```

```
Hello World!
HELLO WORLD!
```

Metodi e dot notation

- `upper()` si scrive **dopo** la stringa, separato da un punto (dot notation): è un **metodo**
- La stringa originale `s1` non viene modificata; `s2` contiene la versione maiuscola
- Cambiano solo le lettere minuscole: spazi, numeri, simboli e maiuscole restano invariati

Python fornisce anche altri tipi di dati: li vedremo in seguito



sAIfer Lab

Joint lab on Safety and Security of AI

Input e output

print(), input(), conversioni con int(), float() e str()

Input e output

- Python offre istruzioni per inviare informazioni in uscita (**output**) e acquisirle dall'esterno (**input**)
- Canali standard: **tastiera** e **schermo**
- Si possono usare tutte le periferiche di I/O a disposizione

Primo esempio: `print()` e `input()`

```
print('ciao, inserisci qualcosa da tastiera')
# scrive sullo schermo

a = input()
# acquisisce da tastiera e inserisce
# il valore nella variabile

print('Ciao, hai scritto', a)
# scrive sullo schermo
```

Stringhe e variabili in `print()`

- `'Ciao, hai scritto'` è una **stringa**: `print()` la visualizza esattamente così
- `a` è una **variabile**, scritta **senza apici**: `print()` mostra il valore memorizzato in `a`
- Con `print('a')` verrebbe stampato il carattere `a`, non il valore della variabile

`input()` con messaggio

Il messaggio per l'utente si può passare direttamente a `input()`, senza una `print()` separata:

```
nome = input('Come ti chiami? ')
```

equivale a

```
print('Come ti chiami?')
nome = input()
```

- Differenza: il cursore resta sulla riga del messaggio, quindi conviene lasciare uno **spazio finale**

Ancora su `print()` (1/2)

- Gli argomenti di `print()` sono una **sequenza di elementi separati da virgola**
- Ogni elemento è un **valore** o qualcosa che **produce un valore**:
  - una **stringa**
  - un **numero**
  - una **variabile**
  - un'**espressione** (per ora, espressioni matematiche)

Ancora su `print()` (2/2)

- Le **stringhe** vengono stampate così come sono
- Le **espressioni matematiche** vengono prima calcolate, poi stampate
- Le **variabili** vengono sostituite con il loro valore
- Tra gli elementi, `print()` inserisce automaticamente uno **spazio**
- Alla fine, `print()` va automaticamente **a capo**

`print()` offre molte altre opzioni: formattazione di testo e numeri, modalità di stampa

`print()`: esempio

```
a = 10.1
b = 2
s = "Buon giorno!"

print(s,"Il risultato è", a*b)
```

```
Buon giorno! Il risultato è 20.2
```

Riferimento ufficiale: Python Tutorial – Input and Output

Non tutte le operazioni sono possibili

```
print('ciao, inserisci un numero!')
a = input() # acquisisce da tastiera
b = a/2  # ERRORE! a è una stringa

print('hai inserito', a)
print('la metà è', b)
```

- `input()` restituisce sempre una **stringa**, anche se l'utente digita un numero: occorre convertirla
- I messaggi d'errore dell'interprete guidano verso la correzione (**anche all'esame!**)

Altro esempio: cosa succede?

```
print('ciao, inserisci un numero!')
a = input()

print(a+a)
```

Le funzioni `int()` e `float()`

- Convertono una stringa in intero o in float, **se la conversione è possibile**
- `float` significa "virgola mobile": il numero è rappresentato con **mantissa** ed **esponente**, come in $3.14 = 314 \cdot 10^{-2}$
- Nel calcolatore le potenze sono in base 2, non in base 10

I numeri reali sono approssimati

- I numeri reali possono essere rappresentati solo in maniera **approssimata** (perché?)

```
print(1-(1/3+1/3+1/3))

print(1-1/3-1/3-1/3)
```

```
0.0

1.110223024625156e-16
```

- Espressioni matematicamente equivalenti, risultati diversi: con i float **l'ordine delle operazioni** influisce sugli arrotondamenti

Arrotondare con `round()`

A causa dell'approssimazione, alcuni calcoli danno risultati "strani", ad esempio con gli importi in euro:

```
prezzo = 1.4 * 3
print(prezzo)                 # 4.199999999999999
print(round(prezzo, 2))       # 4.2 (2 cifre decimali)
print(round(3.7))             # 4 (senza 2° argomento)
```

- `round()` serve per **mostrare** il risultato in modo leggibile: il calcolo interno resta approssimato

`round()` e i valori che terminano in `.5`

- I valori che terminano esattamente in `.5` vengono arrotondati al numero **pari** più vicino

```
round(2.5)    # 2
round(3.5)    # 4
```

Input con conversione

```
print('ciao, inserisci un numero!')

a = input() # acquisisce da tastiera
a = int(a) # oppure float(a) - converte in numero

b = a/2

print('hai inserito', a)
print('la metà è', b)

# alternativa: input e converto inline
# a = int(input())
```

- Con un messaggio: `eta = int(input('Quanti anni hai? '))`

Non sempre le conversioni sono possibili

```
a = "12"
b = "13.55"
c = "ciao"

a1 = int(a)    # 12 rappresentato come intero
a2 = float(a)  # 12.0 rappresentato come float

b1 = int(b)    # ERRORE: conversione a intero
b2 = float(b)  # 13.55 rappresentato come float

c1 = int(c)    # ERRORE: conversione a intero
c2 = float(c)  # ERRORE: conversione a float
```

Conversione inversa: la funzione `str()`

- `str()` converte un numero in stringa: serve, ad esempio, per concatenare un numero e del testo con `+`

```
eta = 20
print('Hai ' + eta + ' anni')         # ERRORE
# TypeError: can only concatenate str (not "int") to str

print('Hai ' + str(eta) + ' anni')    # Hai 20 anni
print('Hai', eta, 'anni')             # Hai 20 anni
```

Primi esempi di programma

```
# il mio primo programma
print("come ti chiami?")
nome_utente = input()
stringa_finale = 'CIAO! ' + nome_utente

print(stringa_finale)
```

Secondo programma

```
# Secondo programma
print('dammi primo numero')
v1_s = input() # stringa
print('dammi secondo numero')
v2_s = input()

v1 = float(v1_s) # converto la stringa in float
v2 = float(v2_s)

risultato = v1 + v2

print('ecco la somma!')
print(risultato)
```



sAIfer Lab

Joint lab on Safety and Security of AI

Tipi di dato

type(), rappresentazione dei float, memoria e id(), keyword

Tipi di dato

- Python gestisce diversi tipi di dati (intero, float, stringa…), ognuno archiviato e manipolato in modo specifico
- Python è **dinamicamente tipizzato**: il tipo non si dichiara, lo deduce dal valore assegnato

Il tipo di dato determina:

- come i dati vengono memorizzati
- quali operazioni si possono eseguire
- come Python li gestisce internamente

La funzione `type()`

- Restituisce il tipo (la classe) dell'oggetto passato come argomento

```
a = 10
b = 11.11
c = 'hello'

print(a, ' tipo di dato: ', type(a))
print(b, ' tipo di dato: ', type(b))
print(c, ' tipo di dato: ', type(c))
```

```
10  tipo di dato:  <class 'int'>
11.11  tipo di dato:  <class 'float'>
hello  tipo di dato:  <class 'str'>
```

Rappresentazione approssimata dei float

- I numeri reali possono avere infinite cifre decimali: la rappresentazione con `float` è necessariamente approssimata

```
a = 1-1/3-1/3 -1/3
print(a)    # teoricamente 0
print(1/a)  # teoricamente impossibile
```

Dove vengono memorizzati i dati?

- La variabile contiene un riferimento all'area di memoria del dato: la funzione `id()` restituisce questo riferimento

```
a = 10
# memorizza 10 in memoria:
# a è un riferimento a quell'area di memoria
b = a
# nuovo riferimento alla stessa area di memoria
print(hex(id(a)), hex(id(b)))
```

- `hex()` converte l'indirizzo in base 16: le due variabili hanno lo **stesso indirizzo**

Cosa restituisce `id()`

- Un valore **univoco** per ogni oggetto del programma
- In CPython, l'implementazione più diffusa, oggi corrisponde all'indirizzo di memoria dell'oggetto
- Ai fini del corso conta che sia un **riferimento univoco**, non che sia il vero indirizzo

docs.python.org/it/3/library/functions.html#id

Cambiare il valore cambia il riferimento

- La variabile **non** contiene il valore, ma solo un riferimento: un meccanismo diverso da quello del linguaggio C

```
a = 1
print(hex(id(a)))
# a si riferisce all'area
# di memoria di 1

a = a +1
print(hex(id(a)))
# 2 è in una nuova area:
# a si riferisce a quella
```

```
0x1030cefa8
0x1030cefc8
```

Keyword

- Le parole chiave (**keyword**) sono termini riservati dal linguaggio, con un significato specifico
- Non possono essere usate come nomi di variabili o funzioni
- L'insieme delle keyword è fissato per ciascuna versione di Python

Elenco delle keyword

```
>>> help("keywords")
```

```
False       class       from        or
None        continue    global      pass
True        def         if          raise
and         del         import      return
as          elif        in          try
assert      else        is          while
async       except      lambda      with
await       finally     nonlocal    yield
break       for         not
```

- Non usatele come nomi di variabili!

Errore subdolo

```
print = 10
```

- `print` diventa il **nome di una variabile**: la funzione `print()` non si può usare fino al riavvio dell'interprete
- Lo stesso vale per altri nomi predefiniti: `input`, `len`, `type`
- Le keyword vere non si possono usare come nomi: `if = 10` produce un errore di sintassi



sAIfer Lab

Joint lab on Safety and Security of AI

Oggetti e metodi

Dato + comportamento · valore e identità: == e is · l'oggetto None

Oggetti: dato + comportamento

- In Python tutti i valori assegnati a una variabile sono **oggetti**
- Un oggetto contiene un **valore** e offre **operazioni** su quel valore: i **metodi**
- I metodi si richiamano con la **dot notation**: `s.upper()` restituisce `s` in maiuscolo

Variabili e oggetti

- Un oggetto è un valore e anche **un insieme di funzionalità** che sanno operare su quel valore
- La variabile **non contiene direttamente il dato** (come in C), ma un riferimento all'area di memoria dell'oggetto
- Diremo spesso "la variabile contiene l'oggetto": è **un'approssimazione**

Valore e identità: == e `is`

== valore

Verifica se due oggetti hanno lo **stesso valore**, anche se sono oggetti distinti

`is` identità

Verifica se due variabili fanno riferimento allo **stesso oggetto** in memoria

- Per confrontare valori si usa quasi sempre `==`
- `is` serve più raramente: per verificare se due variabili puntano **esattamente allo stesso oggetto**

== e is: stesso oggetto

```
a = 10
b = a

print(a == b)   # True
print(a is b)   # True
```

```
True
True
```

- Entrambi `True`: `a` e `b` si riferiscono allo **stesso oggetto**

L'oggetto `None`

- Indica che una variabile **non ha ancora un valore significativo**, o che una funzione **non restituisce alcun risultato**
- È un **singleton**: in tutto il programma esiste una sola istanza di `None`
- Per verificare se una variabile vale `None` è buona pratica usare `is`

`None` e l'operatore `is`

```
x = None

print(x is None)  # True

x = 1 + 2
print(x is None)  # False
```

```
True
False
```

- `x is None` verifica se `x` fa riferimento all'unica istanza di `None`



sAIfer Lab

Joint lab on Safety and Security of AI

Istruzioni

Istruzioni, flusso di esecuzione sequenziale, esempi ed esercizi

Cos'è un'istruzione

- Un'**istruzione** (statement) è un'unità eseguibile di codice che Python può interpretare ed eseguire
- Può includere parole chiave, operatori e funzioni

Esempi: assegnazione di un valore a una variabile, stampa di un risultato

```
a = 10
b = input()
c = 2 * a
print('Risultato', c)
```

Flusso di esecuzione sequenziale

- Python esegue le istruzioni in modo **sequenziale**: dall'alto verso il basso, nell'ordine in cui sono scritte
- L'ordine delle istruzioni determina il comportamento del programma e i risultati
- Più avanti: **condizioni**, **cicli** e **funzioni** per alterare il flusso
  - eseguire istruzioni solo in certe situazioni
  - ripeterle più volte
  - organizzarle in blocchi riutilizzabili



sAIfer Lab

Joint lab on Safety and Security of AI

Esempi ed esercizi

Programmi con input, conversioni e calcoli

1 · costo_colazione_1

- Chiedere all'utente il **numero di paste** mangiate e il **costo unitario**
- Calcolare e stampare il **costo finale**
- Stampare anche il **tipo di dato** e l'**indirizzo di memoria**

`costo_colazione_1`: soluzione (1/2)

```
print('quante paste hai mangiato?')
stringa_numero_paste = input()
numero_paste = int(stringa_numero_paste)
print('costo unitario?')
stringa_costo_unitario = input()
costo_unitario = float(stringa_costo_unitario)
```

`costo_colazione_1`: soluzione (2/2)

```
print('tipo della var stringa_numero_paste:',
      type(stringa_numero_paste))
print('tipo della var numero_paste:',
      type(numero_paste))
print('tipo della var costo_unitario:',
      type(costo_unitario))
print('locazione di memoria della var costo_unitario:',
      hex(id(costo_unitario)))

costo_totale = numero_paste * costo_unitario
print('costo totale', costo_totale)
```

2 · `hello_1`

- Chiedere all'utente **nome** e **cognome** e salutarlo stampando "Ciao" seguito da nome e cognome
- **Variante**: stampare il numero totale di caratteri di nome e cognome

Suggerimento: `len(s)` restituisce la lunghezza di una stringa

3–4 · `rettangolo_1`, `triangolo_1`

3 · rettangolo_1

Chiedere all'utente la lunghezza dei lati di un rettangolo e stampare **area** e **perimetro**

4 · triangolo_1

Chiedere all'utente base e altezza di un triangolo e stampare l'**area**

5 · triangolo_2

- Chiedere all'utente l'**area** di un triangolo rettangolo isoscele e stampare il **perimetro**

Suggerimento: la radice quadrata si calcola con

```
import math
math.sqrt(x)  # radice quadrata di x
```

6–7 · `triangolo_3`, `eq_1`

6 · triangolo_3

Chiedere all'utente la lunghezza dei lati di un **triangolo** e stampare area e perimetro

7 · eq_1

Chiedere i coefficienti dell'equazione $a \cdot x^2 + b \cdot x + c = 0$ e trovare le **radici**

Si assuma che le radici siano reali e distinte

8 · conversione_4bit_1

- Chiedere all'utente le **4 cifre** di un numero intero in base 2 e stampare il valore in base 10
- Calcolare anche il **prodotto per 2** e la **divisione intera per 2** del numero
- Senza usare direttamente prodotto e divisione: sfruttare lo **shift** a destra e a sinistra